Programs

Computer *programs*, known as *software*, are instructions to the computer.

You tell a computer what to do through programs. Without programs, a computer is an empty machine. Computers do not understand human languages, so you need to use computer languages to communicate with them.

Programs are written using programming languages.

Programming Languages

Machine Language Assembly Language High-Level Language

Machine language is a set of primitive instructions built into every computer. The instructions are in the form of binary code, so you have to enter binary codes for various instructions. Program with native machine language is a tedious process. Moreover the programs are highly difficult to read and modify. For example, to add two numbers, you might write an instruction in binary like this:

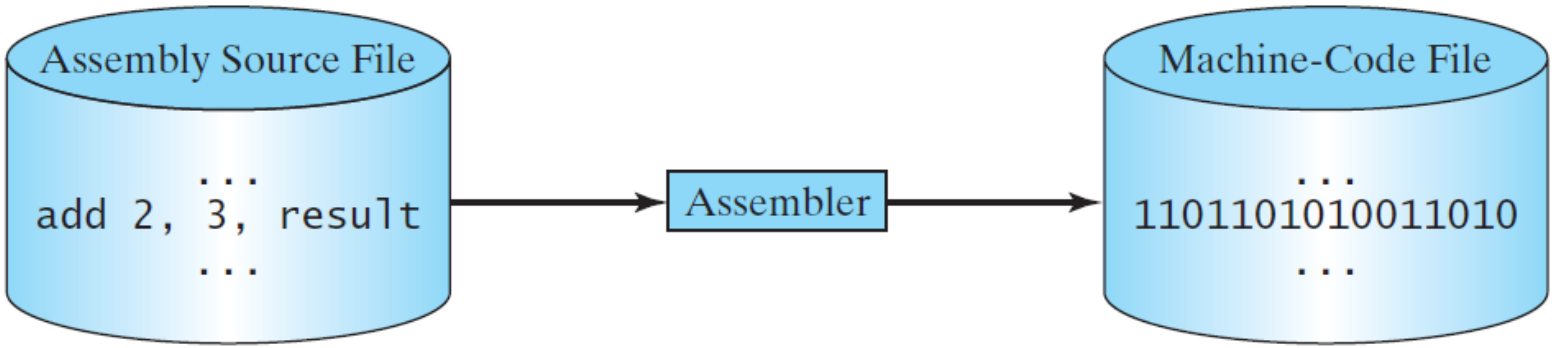
```
1101101010011010
```

Programming Languages

Machine Language Assembly Language High-Level Language

Assembly languages were developed to make programming easy. Since the computer cannot understand assembly language, however, a program called assembler is used to convert assembly language programs into machine code. For example, to add two numbers, you might write an instruction in assembly code like this:

ADDF3 R1, R2, R3



Programming Languages

Machine Language Assembly Language High-Level Language

The high-level languages are English-like and easy to learn and program. For example, the following is a high-level language statement that computes the area of a circle with radius 5:

```
area = 5 * 5 * 3.1415;
```

Popular High-Level Languages

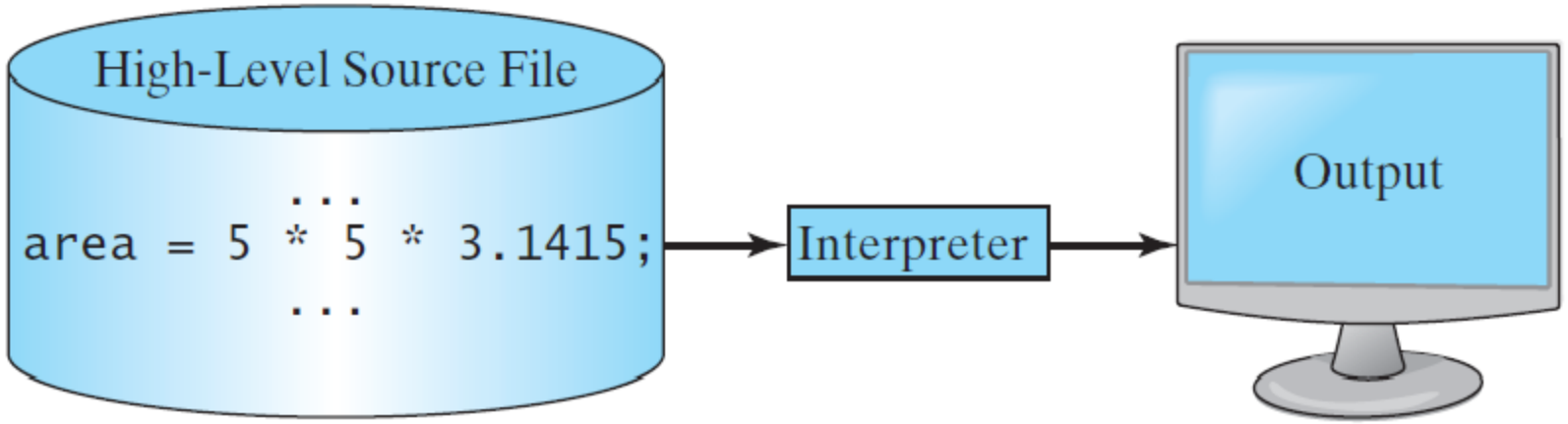
Language	Description
Ada	Named for Ada Lovelace, who worked on mechanical general-purpose computers. The Ada language was developed for the Department of Defense and is used mainly in defense projects.
BASIC	Beginner's All-purpose Symbolic Instruction Code. It was designed to be learned and used easily by beginners.
C	Developed at Bell Laboratories. C combines the power of an assembly language with the ease of use and portability of a high-level language.
C++	C++ is an object-oriented language, based on C.
C#	Pronounced "C Sharp." It is a hybrid of Java and C++ and was developed by Microsoft.
COBOL	COmmon Business Oriented Language. Used for business applications.
FORTRAN	FORmula TRANslation. Popular for scientific and mathematical applications.
Java	Developed by Sun Microsystems, now part of Oracle. It is widely used for developing platform-independent Internet applications.
Pascal	Named for Blaise Pascal, who pioneered calculating machines in the seventeenth century. It is a simple, structured, general-purpose language primarily for teaching programming.
Python	A simple general-purpose scripting language good for writing short programs.
Visual Basic	Visual Basic was developed by Microsoft and it enables the programmers to rapidly develop graphical user interfaces.

Interpreting/Compiling Source Code

A program written in a high-level language is called a *source program* or *source code*. Because a computer cannot understand a source program, a source program must be translated into machine code for execution. The translation can be done using another programming tool called an *interpreter* or a *compiler*.

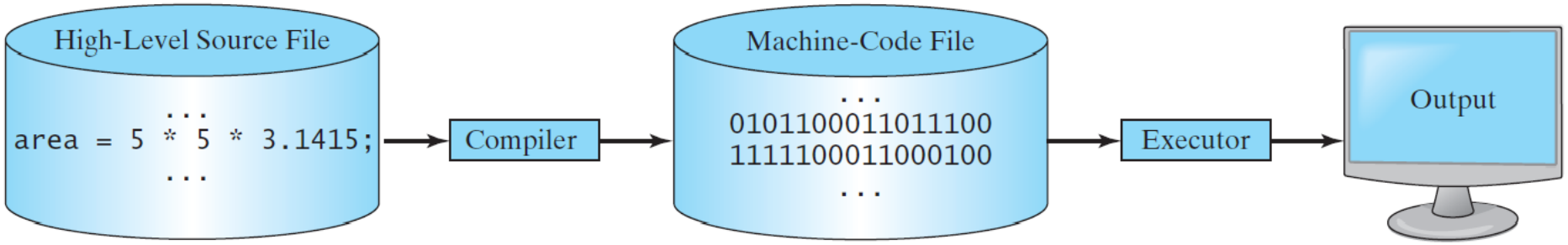
Interpreting Source Code

An interpreter reads one statement from the source code, translates it to the machine code or virtual machine code, and then executes it right away, as shown in the following figure. Note that a statement from the source code may be translated into several machine instructions.



Compiling Source Code

A compiler translates the entire source code into a machine-code file, and the machine-code file is then executed, as shown in the following figure.



Java

- What do you know about Java?

A Little Bit of History

- 1950: FORTRAN
- 1959: COBOL
  - Half business software still programmed in COBOL
- Mid 1960s: Basic
- 1971: Pascal
  - Structured programming language
- 1970s & Early 1980s: Ada

A Little Bit of History

- 1972: C Programming Language
  - What was there before C?
- 1983: ANSII C
- Early 1980's: C++
  - Object-oriented programming language
  - Examples of other OO languages:
    - Smalltalk
    - Java
- 1991: Java (originally Oak) Came Into Being

A Little Bit of History

- 1993: World Wide Web Became Popular
  - Java was saved!
- 1995: Java Formally Announced
- 2000: C#

Why Java?

- The Java programming language is robust and versatile, enabling developers to:
  - Write software on one platform and run it on another.
  - Create programs to run within a web browser.
  - Develop server-side applications for online forums, stores, polls, processing HTML forms, and more.
  - Write applications for cell phones, two-way pagers, and other consumer devices.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java is partially modeled on C++, but greatly simplified and improved. Some people refer to Java as "C++--" because it is like C++ but with more functionality and fewer negative aspects.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java is inherently object-oriented. Although many object-oriented languages began strictly as procedural languages, Java was designed from the start to be object-oriented. Object-oriented programming (OOP) is a popular programming approach that is replacing traditional procedural programming techniques.

One of the central issues in software development is how to reuse code. Object-oriented programming provides great flexibility, modularity, clarity, and reusability through encapsulation, inheritance, and polymorphism.

Characteristics of Java

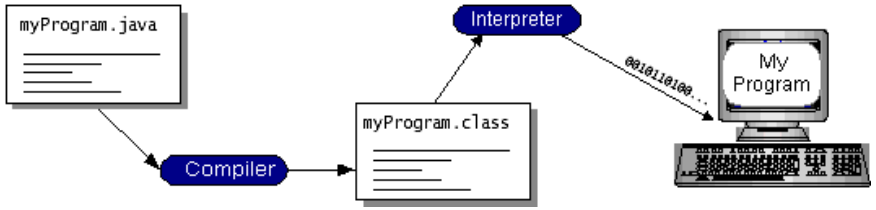
- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Distributed computing involves several computers working together on a network. Java is designed to make distributed computing easy. Since networking capability is inherently integrated into Java, writing network programs is like sending and receiving data to and from a file.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

You need an interpreter to run Java programs. The programs are compiled into the Java Virtual Machine code called bytecode. The bytecode is machine-independent and can run on any machine that has a Java interpreter, which is part of the Java Virtual Machine (JVM).



Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java compilers can detect many problems that would first show up at execution time in other languages.

Java has eliminated certain types of error-prone programming constructs found in other languages.

Java has a runtime exception-handling feature to provide programming support for robustness.

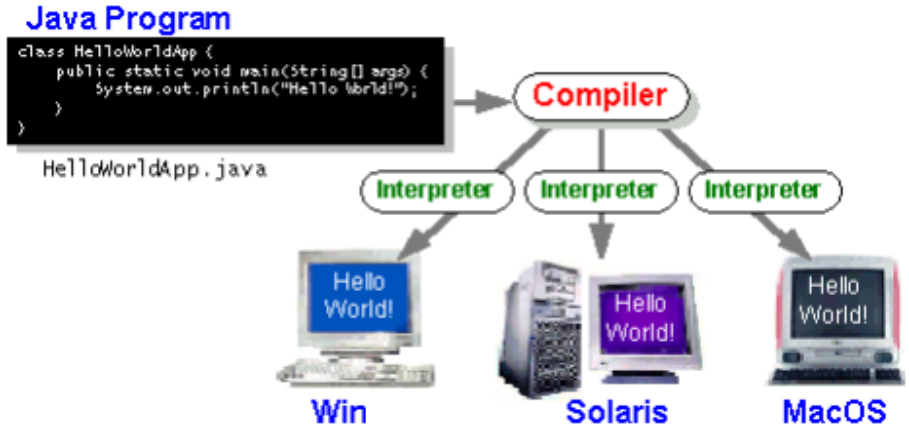
Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java implements several security mechanisms to protect your system against harm caused by stray programs.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic



Write once, run anywhere

With a Java Virtual Machine (JVM), you can write one program that will run on any platform.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Because Java is architecture neutral, Java programs are portable. They can be run on any platform without being recompiled.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java's performance Because Java is architecture neutral, Java programs are portable. They can be run on any platform without being recompiled.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Multithread programming is smoothly integrated in Java, whereas in other languages you have to call procedures specific to the operating system to enable multithreading.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java was designed to adapt to an evolving environment. New code can be loaded on the fly without recompilation. There is no need for developers to create, and for users to install, major new software versions. New features can be incorporated transparently as needed.

JDK Versions

- JDK 1.02 (1995)
- JDK 1.1 (1996)
- JDK 1.2 (1998)
- JDK 1.3 (2000)
- JDK 1.4 (2002)
- JDK 1.5 (2004) a. k. a. JDK 5 or Java 5
- JDK 1.6 (2006) a. k. a. JDK 6 or Java 6
- JDK 1.7 (2011) a. k. a. JDK 7 or Java 7
- JDK 1.8 (2014) a. k. a. JDK 8 or Java 8

JDK Editions

- Java Standard Edition (J2SE)
  - J2SE can be used to develop client-side standalone applications or applets.
- Java Enterprise Edition (J2EE)
  - J2EE can be used to develop server-side applications such as Java servlets, Java ServerPages, and Java ServerFaces.
- Java Micro Edition (J2ME).
  - J2ME can be used to develop applications for mobile devices such as cell phones.

Popular Java IDEs

- NetBeans
- Eclipse

The Java Buzzwords

The key considerations were summed up by the Java team in the following list of buzzwords:

- Simple
- Secure
- Portable
- Object-oriented
- Robust
- Multithreaded
- Architecture-neutral
- Interpreted
- High performance
- Distributed
- Dynamic

The Java Buzzwords

- simple – Java is designed to be easy for the professional programmer to learn and use.
- object-oriented: a clean, usable, pragmatic approach to objects, not restricted by the need for compatibility with other languages.
- Robust: restricts the programmer to find the mistakes early, performs compile-time (strong typing) and run-time (exception-handling) checks, manages memory automatically.
- Multithreaded: supports multi-threaded programming for writing program that perform concurrent computations

The Java Buzzwords

- Architecture-neutral: Java Virtual Machine provides a platform independent environment for the execution of Java byte code
- Interpreted and high-performance: Java programs are compiled into an intermediate representation – byte code:
  - a) can be later interpreted by any JVM
  - b)can be also translated into the native machine code for efficiency.

The Java Buzzwords

- Distributed: Java handles TCP/IP protocols, accessing a resource through its URL much like accessing a local file.
- Dynamic: substantial amounts of run-time type information to verify and resolve access to objects at run-time.
- Secure: programs are confined to the Java execution environment and cannot access other parts of the computer.

The Java Buzzwords

- Portability: Many types of computers and operating systems are in use throughout the world—and many are connected to the Internet.
- For programs to be dynamically downloaded to all the various types of platforms connected to the Internet, some means of generating portable executable code is needed. The same mechanism that helps ensure security also helps create portability.
- Indeed, Java's solution to these two problems is both elegant and efficient.

Data Types

- Java defines eight simple types:
  - 1)byte – 8-bit integer type
  - 2)short – 16-bit integer type
  - 3)int – 32-bit integer type
  - 4)long – 64- bit integer type
  - 5)float – 32-bit floating-point type
  - 6)double – 64-bit floating-point type
  - 7)char – symbols in a character set
  - 8)boolean – logical values true and false

Data Types

- byte: 8-bit integer type.
  - Range: -128 to 127.
  - Example: byte b = -15;
  - Usage: particularly when working with data streams.
- short: 16-bit integer type.
  - Range: -32768 to 32767.
  - Example: short c = 1000;
  - Usage: probably the least used simple type.

Data Types

- int: 32-bit integer type.
  - Range: -2147483648 to 2147483647.
  - Example: int b = -50000;
  - Usage:
    - ✓1) Most common integer type.
    - ✓2) Typically used to control loops and to index arrays.
    - ✓3)Expressions involving the byte, short and int values are promoted to int before calculation.

Data Types

- long: 64-bit integer type.
  - Range: -9223372036854775808 to
  - 9223372036854775807.
  - Example: long l = 10000000000000000;
  - Usage:
    - ✓1) useful when int type is not large enough to hold the desired value
- float: 32-bit floating-point number.
  - Range: 1.4e-045 to 3.4e+038.
  - Example: float f = 1.5;
  - Usage:
    - ✓1) fractional part is needed
    - ✓2) large degree of precision is not required

Data Types

- double: 64-bit floating-point number.
  - Range: 4.9e-324 to 1.8e+308.
  - Example: double pi = 3.1416;
  - Usage:
    - ✓1) accuracy over many iterative calculations
    - ✓2) manipulation of large-valued numbers

Data Types

- char: 16-bit data type used to store characters.
  - Range: 0 to 65536.
  - Example: char c = ‘a’;
  - Usage:
    - ✓1) Represents both ASCII and Unicode character sets;
    - ✓Unicode defines a character set with characters found in (almost) all human languages.
    - ✓2)Not the same as in C/C++ where char is 8-bit and represents ASCII only.

Data Types

- boolean: Two-valued type of logical values.
  - Range: values true and false.
  - Example: boolean b = (1<2);
  - Usage:
    - 1) returned by relational operators, such as 1<2
    - 2)required by branching expressions such as ‘if’ or ‘for’

Variables

- declaration – how to assign a type to a variable
- initialization – how to give an initial value to a variable
- scope – how the variable is visible to other parts of the program
- lifetime – how the variable is created, used and destroyed
- type conversion – how Java handles automatic type conversion
- type casting – how the type of a variable can be narrowed down

Variables

- Java uses variables to store data.
- To allocate memory space for a variable JVM requires:
  - 1) to specify the data type of the variable
  - 2) to associate an identifier with the variable
  - 3)optionally, the variable may be assigned an initial value
- All done as part of variable declaration.

Basic Variable Declaration

- datatype identifier [=value];
- datatype must be
  - A simple datatype
  - User defined datatype (class type)
- Identifier is a recognizable name confirm to identifier rules
- Value is an optional initial value.

Variable Declaration

- We can declare several variables at the same time:

  type identifier [=value][, identifier [=value] …];

  - ✓ Examples:
    - int a, b, c;
    - int d = 3, e, f = 5;
    - byte g = 22;
    - double pi = 3.14159;
    - char ch = 'x';

Variable Scope

- Scope determines the visibility of program elements with respect to other program elements.
- In Java, scope is defined separately for classes and methods:
  - 1) variables defined by a class have a global scope
  - 2) variables defined by a method have a local scope
  - A scope is defined by a block:

```
{
        …
}
```

  - A variable declared inside the scope is not visible outside:

```
{
        int n;
}
n = 1;// this is illegal
```

Variable Lifetime

- Variables are created when their scope is entered by control flow and destroyed when their scope is left
- A variable declared in a method will not hold its value between different invocations of this method.
- A variable declared in a block looses its value when the block is left.
- Initialized in a block, a variable will be re-initialized with every re-entry. Variables lifetime is confined to its scope!

Arrays

- An array is a group of like-typed variables referred to by a common name, with individual variables accessed by their index.
- Arrays are:
  - 1) declared
  - 2) created
  - 3) initialized
  - 4) used
- Also, arrays can have one or several dimensions.

Array Declaration

- Array declaration involves:
  - 1) declaring an array identifier
  - 2) declaring the number of dimensions
  - 3) declaring the data type of the array elements
- Two styles of array declaration:
  - type array-variable[];
  - or
  - type [] array-variable;

Array Creation

- After declaration, no array actually exists.
- In order to create an array, we use the new operator:
  - type array-variable[];
  - array-variable = new type[size];
- This creates a new array to hold size elements of type type, which reference will be kept in the variable array-variable.

Array Indexing

- Later we can refer to the elements of this array through their indexes:
  - array-variable[index]
- The array index always starts with zero!
- The Java run-time system makes sure that all array indexes are in the correct range, otherwise raises a run-time error.

Array Initialization

- Arrays can be initialized when they are declared:

- int monthDays[] = {31,28,31,30,31,30,31,31,30,31,30,31};

- Note:
  - 1) there is no need to use the new operator
  - 2)the array is created large enough to hold all specified elements

Multidimensional Arrays

- Multidimensional arrays are arrays of arrays:

- 1) declaration: `int array[][];`
- 2) creation: `int array = new int[2][3];`
- 3) initialization

```
int array[][] = { {1, 2, 3}, {4, 5, 6} };
```

Operators Types

- Java operators are used to build value expressions.

- Java provides a rich set of operators:
  - 1) assignment
  - 2) arithmetic
  - 3) relational
  - 4) logical
  - 5) bitwise

Arithmetic assignments

+=	v += expr;	v = v + expr ;
-=	v -=expr;	v = v - expr ;
*=	v *= expr;	v = v * expr ;
/=	v /= expr;	v = v / expr ;
%=	v %= expr;	v = v % expr ;

Basic Arithmetic Operators

+	op1 + op2	ADD
-	op1 - op2	SUBSTRACT
*	op1 * op2	MULTIPLY
/	op1 / op2	DIVISION
%	op1 % op2	REMAINDER

Relational operator

==	Equals to	Apply to any type
!=	Not equals to	Apply to any type
>	Greater than	Apply to numerical type
<	Less than	Apply to numerical type
>=	Greater than or equal	Apply to numerical type
<=	Less than or equal	Apply to numerical type

Logical operators

&	op1 & op2	Logical AND
|	op1 | op2	Logical OR
&&	op1 && op2	Short-circuit AND
||	op1 || op2	Short-circuit OR
!	! op	Logical NOT
^	op1 ^ op2	Logical XOR

Bit wise operators

~	~op	Inverts all bits
&	op1 & op2	Produces 1 bit if both operands are 1
|	op1 |op2	Produces 1 bit if either operand is 1
^	op1 ^ op2	Produces 1 bit if exactly one operand is 1
>>	op1 >> op2	Shifts all bits in op1 right by the value of op2
<<	op1 << op2	Shifts all bits in op1 left by the value of op2

Expressions

- An expression is a construct made up of variables, operators, and method invocations, which are constructed according to the syntax of the language, that evaluates to a single value.
- Examples of expressions are in bold below:

```
int number = 0;
anArray[0] = 100;
System.out.println ("Element 1 at index 0: " +
anArray[0]);
int result = 1 + 2; // result is now 3 if(value1 ==
value2)
System.out.println("value1 == value2");
```

Expressions

- The data type of the value returned by an expression depends on the elements used in the expression.
- The expression number = 0 returns an int because the assignment operator returns a value of the same data type as its left-hand operand; in this case, number is an int.
- As you can see from the other expressions, an expression can return other types of values as well, such as boolean or String.
- The Java programming language allows you to construct compound expressions from various smaller expressions as long as the data type required by one part of the expression matches the data type of the other.
- Here's an example of a compound expression: 1 * 2 * 3

Control Statements

- Java control statements cause the flow of execution to advance and branch based on the changes to the state of the program.
- Control statements are divided into three groups:
  - 1) selection statements allow the program to choose different parts of the execution based on the outcome of an expression
  - 2) iteration statements enable program execution to repeat one or more statements
  - 3) jump statements enable your program to execute in a non-linear fashion

Selection Statements

- Java selection statements allow to control the flow of program's execution based upon conditions known only during run-time.
- Java provides four selection statements:
  - 1) if
  - 2) if-else
  - 3) if-else-if
  - 4) switch

Iteration Statements

- Java iteration statements enable repeated execution of part of a program until a certain termination condition becomes true.
- Java provides three iteration statements:
  - 1) while
  - 2) do-while
  - 3) for

Jump Statements

- Java jump statements enable transfer of control to other parts of program.
- Java provides three jump statements:
  - 1) break
  - 2) continue
  - 3) return
- In addition, Java supports exception handling that can also alter the control flow of a program.

Type Conversion

- Size Direction of Data Type
  - – Widening Type Conversion (Casting down)
    - • Smaller Data Type -> Larger Data Type
  - – Narrowing Type Conversion (Casting up)
    - • Larger Data Type -> Smaller Data Type
- • Conversion done in two ways
  - – Implicit type conversion
    - • Carried out by compiler automatically
  - – Explicit type conversion
    - • Carried out by programmer using casting

Type Conversion

- Widening Type Converstion
  - – Implicit conversion by compilerautomatically

byte -> short, int, long, float, double
short -> int, long, float, double
char -> int, long, float, double
int -> long, float, double
long -> float, double
float -> double

Type Conversion

- Narrowing Type Conversion
  - – Programmer should describethe conversion explicitly

byte -> char
short -> byte, char
char -> byte, short
int -> byte, short, char
long -> byte, short, char, int
float -> byte, short, char, int, long
double -> byte, short, char, int, long, float

Type Conversion

- byte and short are always promoted to int
- if one operand is long, the whole expression is promoted to long
- if one operand is float, the entire expression is promoted to float
- if any operand is double, the result is double

Type Casting

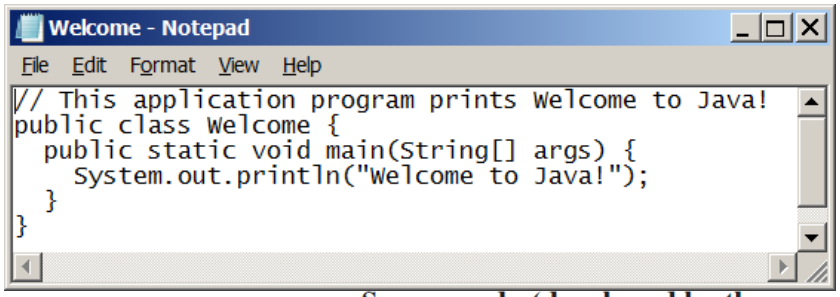
- General form: (targetType) value
  - Examples:
  - 1) integer value will be reduced module bytes range:
    - ✓int i;
    - ✓byte b = (byte) i;
  - 2) floating-point value will be truncated to integer value:
    - ✓float f;
    - ✓int i = (int) f;

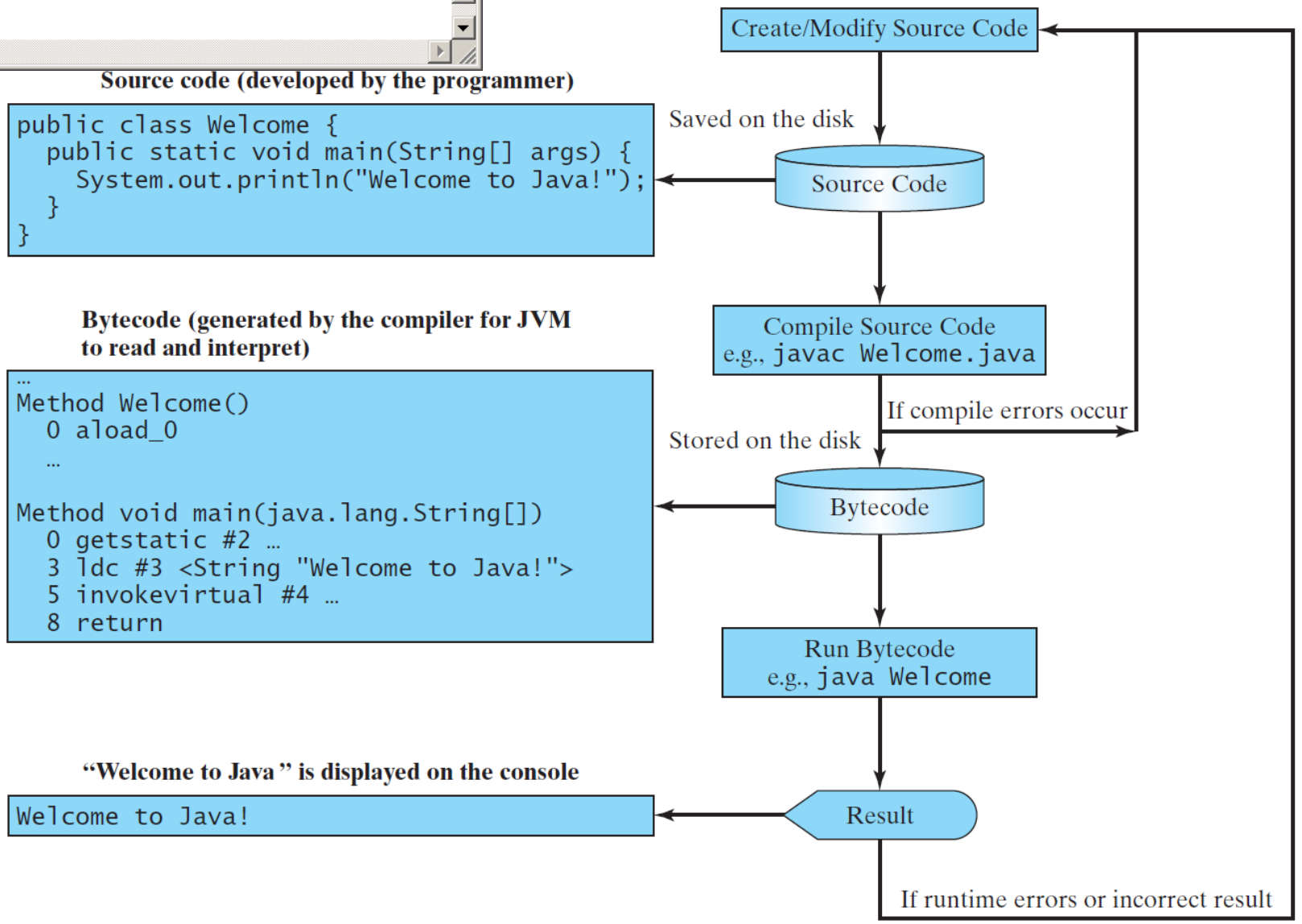
Simple Java Program

A class to display a simple message:

```
class MyProgram
{
        public static void main(String[] args)
        {
                System.out.println(“First Java program.");
        }
}
```

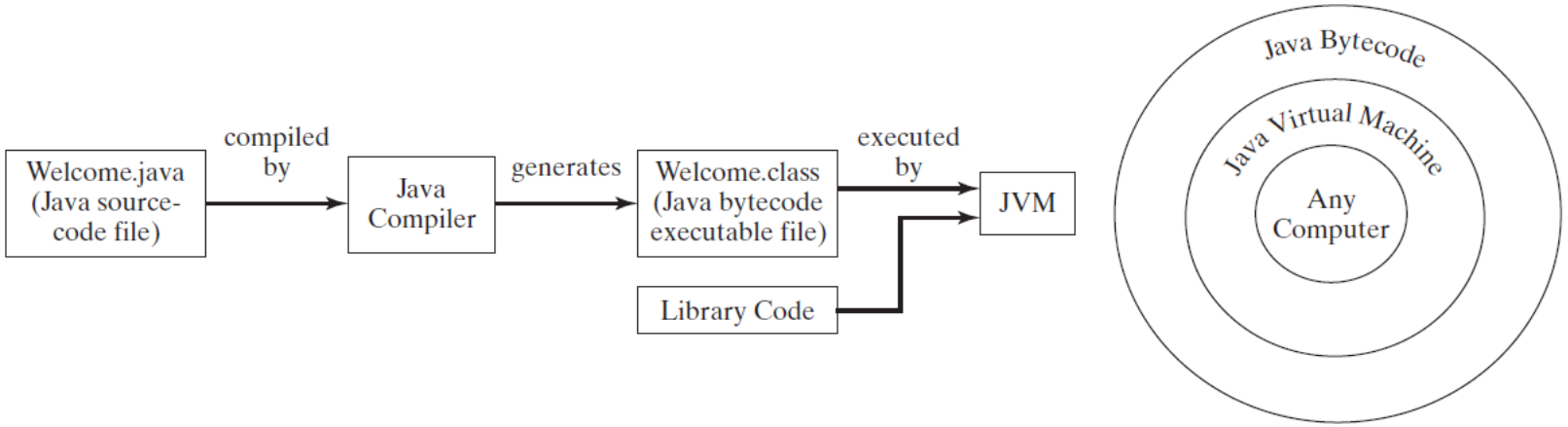
Creating, Compiling, and Running Programs





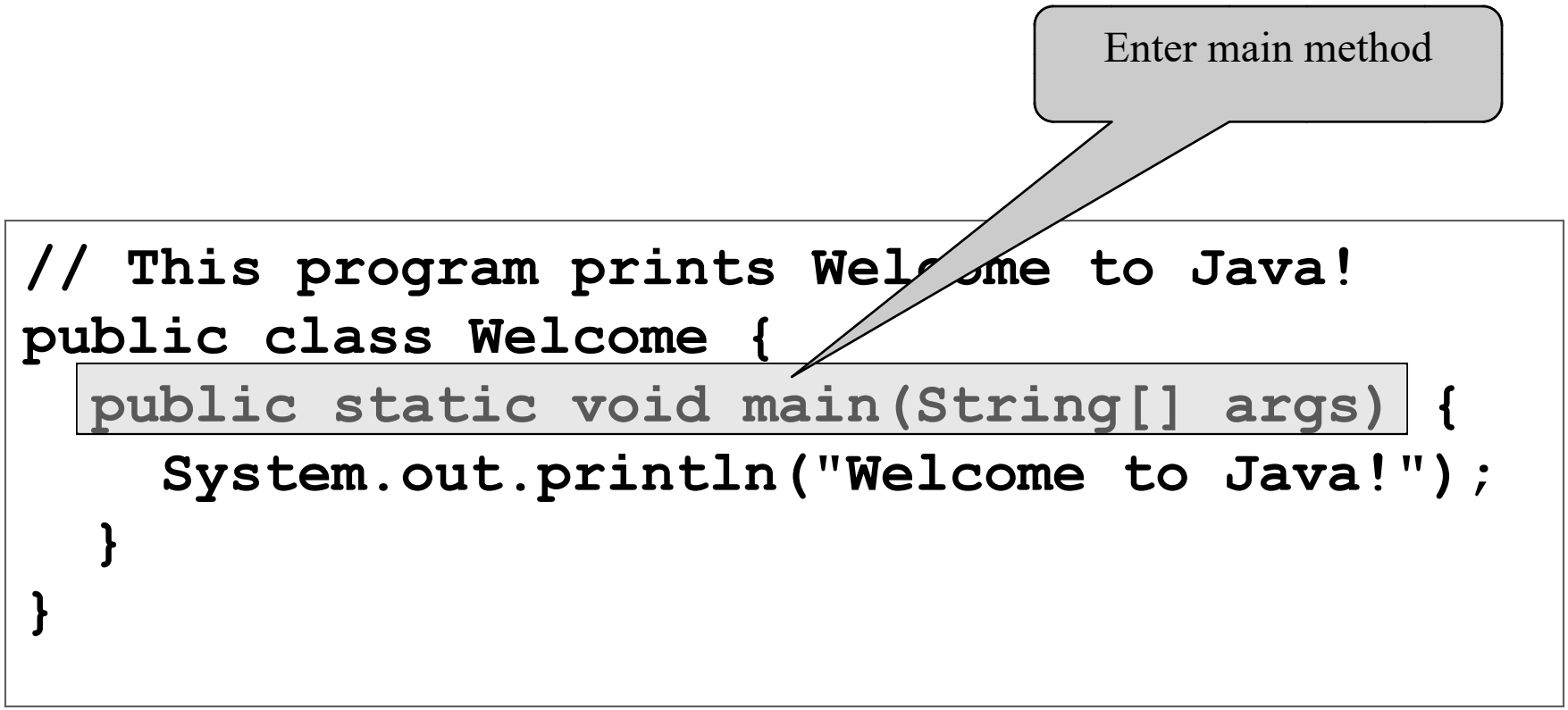
Compiling Java Source Code

You can port a source program to any machine with appropriate compilers. The source program must be recompiled, however, because the object program can only run on a specific machine. Nowadays computers are networked to work together. Java was designed to run object programs on any platform. With Java, you write the program once, and compile the source program into a special type of object code, known as *bytecode*. The bytecode can then run on any computer with a Java Virtual Machine, as shown below. Java Virtual Machine is a software that interprets Java bytecode.



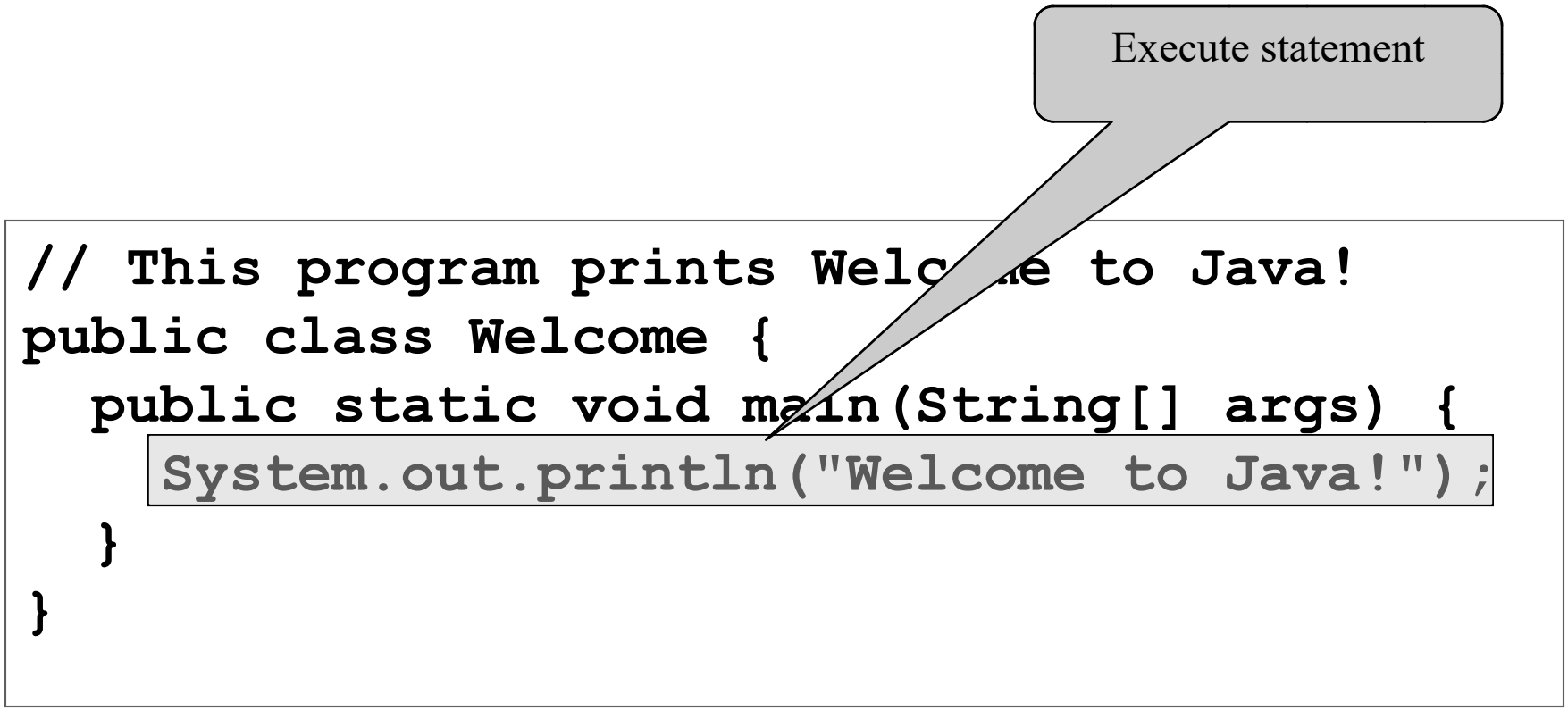
animation

Trace a Program Execution



animation

Trace a Program Execution



animation

Trace a Program Execution

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```



print a message to the console

Java Programs

- Create or write your Java programs using any text editor. It is always best to use a text editor that recognizes Java instructions.
- Save your programs with a .java extension
  - welcome.java
- Compile your java program in the command window by typing
  - javac welcome.java
  - You will need to go to the directory containing your file

Java Programs

- Compiling Java programs creates a new file with the same name and the extension .class
  - welcome.class
- This file contains the bytecode that will be interpreted during the next step
- You run the Java program by typing
  - java welcome
- In this stage the program is loaded into memory and executed

What is an Object?

- Real world objects are things that have:
  - 1) state
  - 2)behavior
  - Example: Mydog:
- state – name, color, breed
- behavior – sitting, barking, waging tail, running
- A software object is a bundle of variables (state) and methods (operations).

What is a Class?

- A class is a blueprint that defines the variables and methods common to all objects of a certain kind.
- Example: ‘Mydog’ is an object of the class Dog.
- An object holds values for the variables defined in the class.
- An object is called an instance of the Class

Object Creation

- A variable is declared to refer to the objects of type/class String:

  String s;

- The value of s is null; it does not yet refer to any object.
- A new String object is created in memory with initial “abc” value:
- String s = new String(“abc”);
- Now s contains the address of this new object.

Object Destruction

- A program accumulates memory through its execution.
- Two mechanisms to free memory that is no longer need by the program:
  - 1) manual – done in C/C++
  - 2) automatic – done in Java
- In Java, when an object is no longer accessible through any variable, it is eventually removed from the memory by the garbage collector.
- Garbage collector is part of the Java Run-Time Environment.

Class

- A basis for the Java language.
- Each concept we wish to describe in Java must be included inside a class.
- A class defines a new data type, whose values are objects:
- A class is a template for objects
- An object is an instance of a class

Class Definition

- A class contains a name, several variable declarations (instance variables) and several method declarations. All are called members of the class.
- General form of a class:

```
class classname {
        type instance-variable-1;
        …
        type instance-variable-n;
        type method-name-1(parameter-list) { … }
        type method-name-2(parameter-list) { … }
        …
        type method-name-m(parameter-list) { … }
}
```

Example: Class Usage

```
class Container {
        double width;
        double height;
        double depth;
}
class ContainerDemo {
public static void main(String args[]) {
        Container mycontnr = new Container();
        double vol;
        mycontnr.width = 10;
        mycontnr.height = 20;
        mycontnr.depth = 15;
        vol = mycontnr.width * mycontnr.height *
mycontnr.depth;
        System.out.println ("Volume is " + vol);
} }
```

Constructor

- A constructor initializes the instance variables of an object.
- It is called immediately after the object is created but before the new operator completes.
  - 1) it is syntactically similar to a method:
  - 2) it has the same name as the name of its class
  - 3)it is written without return type; the default return type of a class
- constructor is the same class When the class has no constructor, the default constructor automatically initializes all its instance variables with zero.

What is a Constructor?

- Constructor is a special method that gets invoked "automatically" at the time of object creation.
- Constructor is normally used for initializing objects with default values unless different values are supplied.
- Constructor has the same name as the class name.
- Constructor cannot return values.
- A class can have more than one constructor as long as they have different signature (i.e., different input arguments syntax).

Defining a Constructor

Like any other method

```
public class ClassName {

   // Data Fields…

   // Constructor
   public ClassName()
   {
      // Method Body Statements initialising Data Fields
   }

   //Methods to manipulate data fields
}
```

Invoking:

There is NO explicit invocation statement needed: When the object creation statement is executed, the constructor method will be executed automatically.

Defining a Constructor: Example

```
public class Counter {
    int CounterIndex;

    // Constructor
    public Counter()
    {
        CounterIndex = 0;
    }
    //Methods to update or access counter
    public void increase()
    {
        CounterIndex = CounterIndex + 1;
    }
    public void decrease()
    {
        CounterIndex = CounterIndex - 1;
    }
    int getCounterIndex()
    {
      return CounterIndex;
    }
}
```

A Counter with User Supplied Initial Value ?

This can be done by adding another constructor method to the class.

```
public class Counter {
    int CounterIndex;

    // Constructor 1
    public Counter()
    {
        CounterIndex = 0;
    }
    public Counter(int InitValue )
    {
        CounterIndex = InitValue;
    }
}

// A New User Class: Utilising both constructors
            Counter counter1 = new Counter();
            Counter counter2 = new Counter (10);
```

Multiple Constructors: Example

```
public class Circle {
        public double x,y,r; //instance variables
    // Constructors
        public Circle(double centreX, double cenreY, double radius) {
                x = centreX;  y = centreY; r = radius;
        }
        public Circle(double radius) { x=0; y=0; r = radius; }
        public Circle() { x=0; y=0; r=1.0; }

     //Methods to return circumference and area
        public double circumference() { return 2*3.14*r; }
        public double area() { return 3.14 * r * r; }
}
```

Method Overloading

- Constructors all have the same name.
- Methods are distinguished by their signature:
  - name
  - number of arguments
  - type of arguments
  - position of arguments
- That means, a class can also have multiple usual methods with the same name.

The New this keyword

this keyword can be used to refer to the object itself.

❖It is generally used for accessing class members (from its own methods) when they have the same name as those passed as arguments.

```
public class Circle {
        public double x,y,r;
    // Constructor
        public Circle (double x, double y, double r) {
                this.x = x;
                this.y = y;
                this.r = r;
            }
        //Methods to return circumference and area
            }
```

What is Garbage Collection?

- Memory Management technique.
- Process of freeing objects.
- No longer referenced by the program.

Why Garbage Collection?

- Free unreferenced objects.
- Combat heap fragmentation.
- Relieves programmer from manual freeing the memory.
- Helps in ensuring program integrity.
- Disadvantages
  - Extra Overhead.
  - Less control over scheduling of CPU time.

Garbage Collection in Java

Purpose

- Find and delete unreachable objects.
- Free space as much as possible.

When??

- GC is under control of JVM.
- One can request but no guarantees.

How??

- Discovery of unreachable objects.
- With the help of Algorithms.

Garbage Collection in Java

Ways for making objects eligible for collection

- Nulling a reference
- Reassigning a reference variable
- Isolating a reference

Forcing garbage collection

- Methods available to perform GC
- Only requests and no demands
- Using Runtime class
- Using static methods like System.gc()

Garbage Collection Algorithms

Basic Approaches

- Reference Counting
- Tracing
- Compacting
- Copying
- Generational
- The Train Algorithm

Binding

- Binding is what happens when a method invocation is bound to an implementation
  - Involves lookup of the method in the class, or one or its parents
  - Both method names and parameters are checked
- Binding can happen at two different times
  - Compile time == static binding
  - Run time == dynamic binding

Dynamic Binding

- Achieved at runtime
  - When the class of an object cannot be determined at compile time
  - Means the JVM (not the compiler) must bind a method call to its implementation
- Instances of a sub-class can be treated as if they were an instance of the parent class
  - Therefore the compiler doesn't know its type, just its base type.

Dynamic Binding Summary

- Whenever a reference refers to an interface or a base class, methods are dynamically bound
- Method implementation determined at runtime
- Dynamic Binding == Polymorphism
- Very powerful OO feature
  - Allows the creation of “frameworks”
- Applications that are implemented around interfaces, but are customised by plugging in different implementations of those interfaces
  - Very extensible

Kinds of Classes

- Top level classes
  - Declared inside package
  - Visible throughout package, perhaps further
  - Normally, although not always, declared in their own file
  - public classes must be defined in their own file
- Nested and inner classes
  - Declared inside class (or method)
  - can be visible only to outer class, or have wider visibility

Kinds of nested/inner classes

- Inner class
  - defined inside another class
  - but each instance of an inner class is transparently associated with an instance of the outer class
  - method invocations can be transparently redirected to outer instance
- Nested class
  - defined inside another class
  - It has unlimited access to its enclosing class members
  - It does not have access to the instance variables or methods of the outer class, unless they are explicitly passed

Inner Classes

Description

Class defined in scope of another class

Property

Can directly access all variables & methods of enclosing class (including private fields & methods)

Example

```
public class OuterClass {
    public class InnerClass {
        ...
    }
}
```

Example of Inner Classes

Motivating Example for Inner and Nested Classes

Let's consider a real-world scenario to make the concepts of inner and nested classes more motivating. We will use the example of an Online Shopping System where we have an outer class representing an Order and inner and nested classes representing Item and OrderStatus.

- The Order class has an orderId, a customer, and a list of Item objects.
- The inner class Item can directly access and modify the Order's list of items.
- The inner class helps in logically grouping the items with the order they belong to.
- The main method creates an Order and adds items to it using the inner class.

Example of Inner Classes

```
class OuterClass {
    private String outerField = "Outer field";
    private static String staticOuterField = "Static Outer Field";

    void outerMethod() {
        System.out.println("Outer Method");
    }

    class InnerClass {
        void display() {
            // Accessing outer class's private field and method
            System.out.println("Accessing: " + outerField);
            outerMethod();
        }
    }

    public static void main(String[] args) {
        // Instantiating inner class
        OuterClass outer = new OuterClass();
        OuterClass.InnerClass inner = outer.new InnerClass();
        inner.display();
    }
}
```

Nested Classes

Declared like a standard inner class, except you say "static class" rather than "class".

For example:

```
class LinkedList {
  static class Node {
    Object head;
    Node tail;
    }
  Node head;
  }
```

Example of Nested Classes

Scenario: Company and Department

Let's consider a Company that has multiple Departments. the Company class contains a static nested class Department. The Department class represents different departments within the company and is independent of any specific instance of Company.

- The Company class need not have any instance variables or methods directly related to departments.
- The static nested class Department represents different departments within the company.
- The Department class has instance variables **name** and **manager** and a method **displayDepartment** to display department details.
- The static nested class can be used to create and manage department instances independently of any specific Company instance.

Example of Nested Classes

```
class OuterClass {
    private static String staticOuterField = "Static Outer Field";

    static void staticOuterMethod() {
        System.out.println("Static Outer Method");
    }

    static class StaticNestedClass {
        void display() {
            // Accessing static members of the outer class
            System.out.println("Accessing: " + staticOuterField);
            staticOuterMethod();
        }
    }

    public static void main(String[] args) {
        // Instantiating static nested class
        OuterClass.StaticNestedClass nested = new OuterClass.StaticNestedClass();
        nested.display();
    }
}
```

Example: Constructor

```
class Container {
    double width;
    double height;
    double depth;
    Container() {
       System.out.println("Constructing Container");
       width = 10; height = 10; depth = 10;
    }
    doublevolume() {
    return width * height * depth;
    }
}
```

Parameterized Constructor

```
class Container {
    double width;
    double height;
    double depth;
    Container(double w, double h, double d) {
       width = w; height = h; depth = d;
    }
    double volume()
    { return width * height * depth;
    }
}
```

Methods

- General form of a method definition:

  type name(parameter-list) {
  … return value;
  …
  }

- Components:

  1)type - type of values returned by the method. If a method does not return any value, its return type must be void.
  2) name is the name of the method
  3) parameter-list is a sequence of type-identifier lists separated by commas
  4) return value indicates what value is returned by the method.

Example: Method

❖Classes declare methods to hide their internal data structures, as well as for their own internal use: Within a class, we can refer directly to its member variables:

```
class Container {
        double width, height, depth;
        void volume() {
                System.out.print("Volume is ");
                System.out.println(width * height * depth);
        }
}
```

Parameterized Method

- Parameters increase generality and applicability of a method:
- 1) method without parameters
  - int square() { return 10*10; }
- 2) method with parameters
  - int square(int i) { return i*i; }
- Parameter: a variable receiving value at the time the method is invoked.

Java Modifier Types

1. Java Access Modifiers

❖Java provides a number of access modifiers to set access levels for classes, variables, methods and constructors.

❖The four access levels are:

- Visible to the package, the default. No modifiers are needed.
- Visible to the class only (private).
- Visible to the world (public).
- Visible to the package and all subclasses (protected).

Java Modifier Types

- Default Access Modifier - No keyword:
- Default access modifier means we do not explicitly declare an access modifier for a class, field, method, etc.
- A variable or method declared without any access control modifier is available to any other class in the **same package**.

- Example: Variables and methods can be declared without any modifiers, as in the following example:

```
String version ="1.5.1";
boolean processOrder(){
        return true;
}
```

Java Modifier Types

- Private Access Modifier - private:
- Methods, Variables and Constructors that are declared private can only be accessed within the declared class itself.
- Variables that are declared private can be accessed outside the class if public getter methods are present in the class.
- Using the private modifier is the main way that an object encapsulates itself and hides data from the outside world.
- Example:
- The following class uses private access control:

```
public class Logger{
private String format;
public String getFormat(){
        return this.format;
}
public void setFormat(String format){
        this.format = format;
}
}
```

Java Modifier Types

- Public Access Modifier - public:
- A class, method, constructor, interface etc declared public can be accessed from any other class. Therefore fields, methods, blocks declared inside a public class can be accessed from any class belonging to the Java Universe. However if the public class we are trying to access is in a different package, then the public class still need to be imported.
- Because of class inheritance, all public methods and variables of a class are inherited by its subclasses.
- Example:

  The following function uses public access control:

```
public static void main(String[] arguments){
// ...
}
```

Java Modifier Types

- Protected Access Modifier - protected:
- Variables, methods and constructors which are declared protected in a superclass can be accessed only by the subclasses in other package or any class within the package of the protected members' class.
- Protected access gives the subclass a chance to use the helper method or variable, while preventing a nonrelated class from trying to use it.
- Example:
- The following parent class uses protected access control, to allow its child class overrideopenSpeaker() method:
  - class AudioPlayer{
  - protected boolean openSpeaker(Speaker sp){
    - // implementation details
  - }
  - }
  - class StreamingAudioPlayer{
  - boolean openSpeaker(Speaker sp){
    - // implementation details
  - }
  - }

Java Modifier Types

- **Access Control and Inheritance**:
- The following rules for inherited methods are enforced:
  - Methods declared public in a superclass also must be public in all subclasses.
  - Methods declared protected in a superclass must either be protected or public in subclasses; they cannot be private.
  - Methods declared without access control (no modifier was used) can be declared more private in subclasses.
  - Methods declared private are not inherited at all, so there is no rule for them.

Comparison of Types of Access Modifier in Java

Access Modifier in Java	Same class	Same package non-subclass	Same package subclassDifferent package subclassDifferent package non-subclass		
Private	Yes	No	No	No	No
Default	Yes	Yes	Yes	No	No
Public	Yes	Yes	Yes	Yes	
Protected	Yes	Yes	Yes	Yes	No

Members of Java that can be assigned with the Java Access Modifiers

Members of JAVA	Private	Default	Protected	Public
Class	No	Yes	No	Yes
Variable	Yes	Yes	Yes	Yes
Method	Yes	Yes	Yes	Yes
Constructor	Yes	Yes	Yes	Yes
Interface	No	Yes	No	Yes

Java Modifier Types

- Non Access Modifiers:
- Java provides a number of non-access modifiers to achieve many other functionality.
  - The static modifier for creating class methods and variables
  - The final modifier for finalizing the implementations of classes, methods, and variables.
  - The abstract modifier for creating abstract classes and methods.
  - The synchronized and volatile modifiers, which are used for threads.
- To use a modifier, you include its keyword in the definition of a class, method, or variable.

Static

Purpose: Used to create class-level variables and methods, which are shared among all instances of the class.
Example:

```
class StaticExample {
    static int staticVariable = 10; // static variable

    static void staticMethod() { // static method
        System.out.println("Static method called.");
    }

    public static void main(String[] args) {
        // Accessing static variable and method without creating an instance
        System.out.println(StaticExample.staticVariable);
        StaticExample.staticMethod();
    }
}
```

staticVariable and staticMethod can be accessed without creating an instance of StaticExample.

Final

Purpose: Used to declare constants, prevent method overriding, and prevent inheritance.

Example:

```
final class FinalClass { // final class
    final int finalVariable = 100; // final variable

    final void finalMethod() { // final method
        System.out.println("Final method called.");
    }
}

class FinalExample { // This will cause a compile error
    // void finalMethod() { } // This will cause a compile error
    public static void main(String[] args) {
        FinalClass obj = new FinalClass();
        System.out.println(obj.finalVariable);
        obj.finalMethod();
    }
}
```

FinalClass cannot be extended. finalVariable value cannot be changed. finalMethod cannot be overridden.

Abstract

Purpose: Used to define abstract classes and methods that must be implemented by subclasses.
Example:

```
abstract class AbstractClass {
    abstract void abstractMethod(); // abstract method

    void concreteMethod() {
        System.out.println("Concrete method called.");
    }
}

class ConcreteClass extends AbstractClass {
    @Override
    void abstractMethod() {
        System.out.println("Abstract method implemented.");
    }
}

public class AbstractExample {
    public static void main(String[] args) {
        ConcreteClass obj = new ConcreteClass();
        obj.abstractMethod();
        obj.concreteMethod();
    }
}
```

AbstractClass cannot be instantiated directly.
abstractMethod must be implemented by the subclass ConcreteClass.

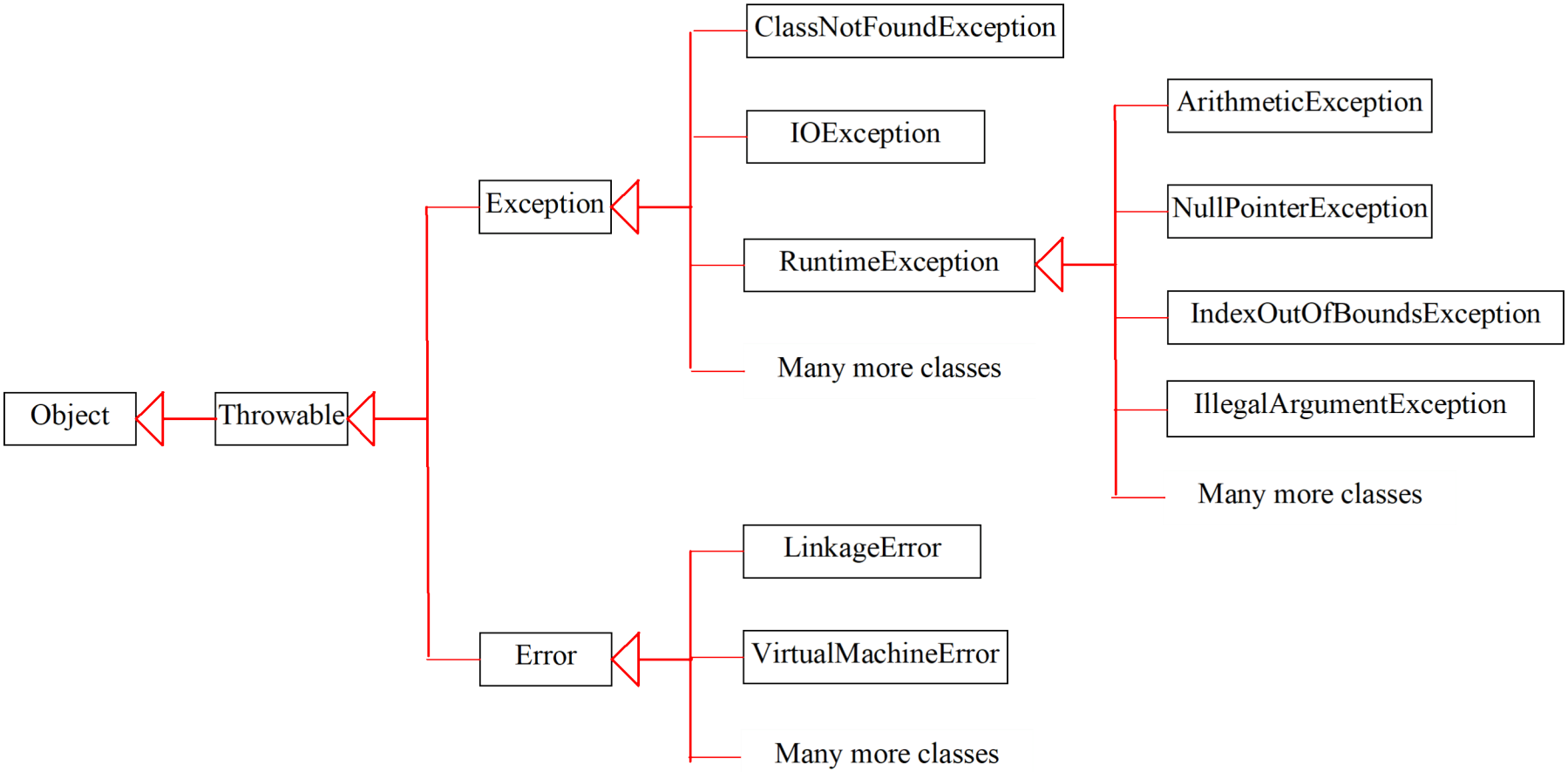
Exception Handling in Java

- Exception Handling
- Advantage of Exception Handling
- Hierarchy of Exception classes

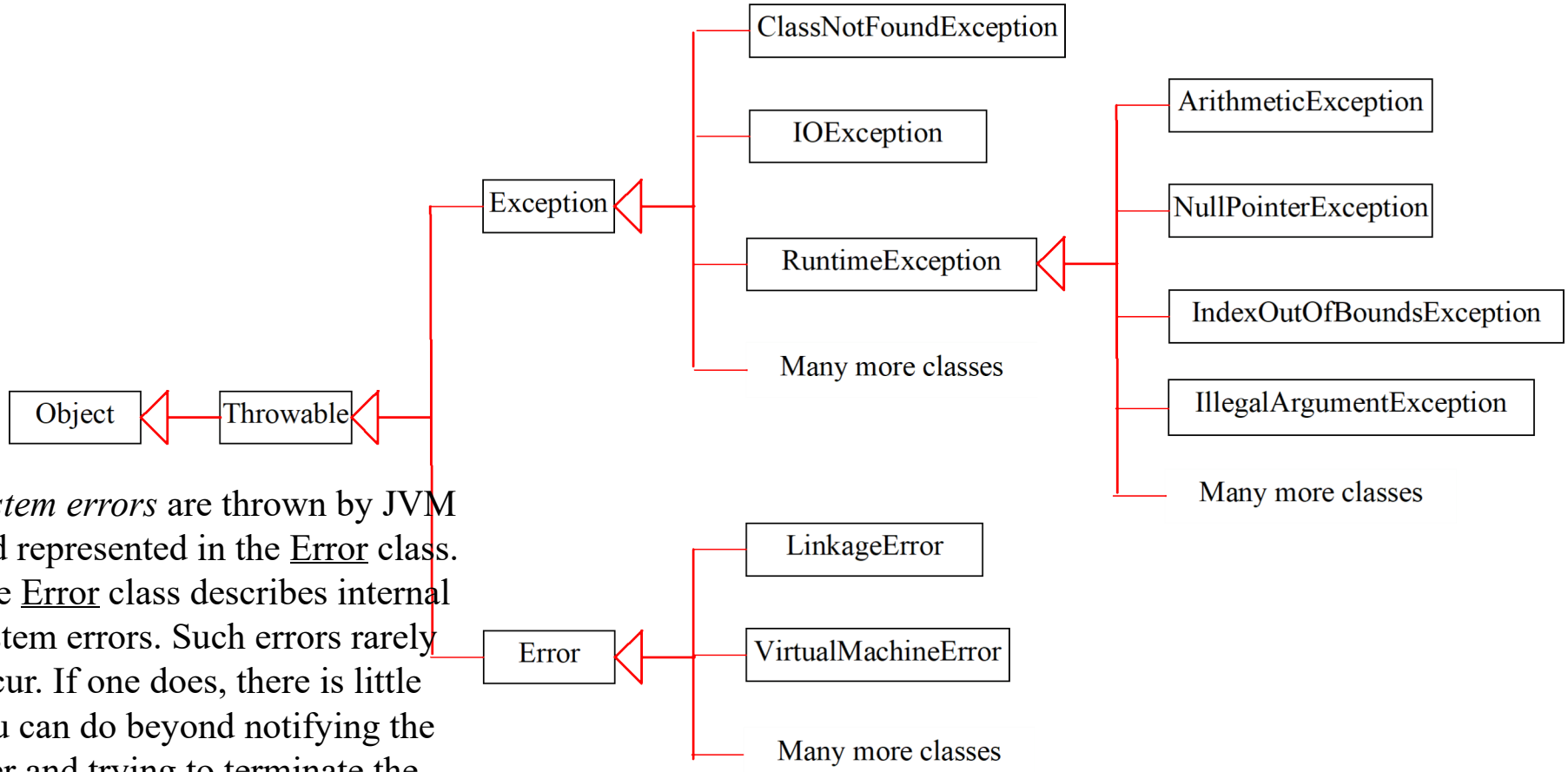
Types of Exception

1. Checked Exception
2. Unchecked Exception
3. Error
5. Scenarios where exception may occur

Exception Types



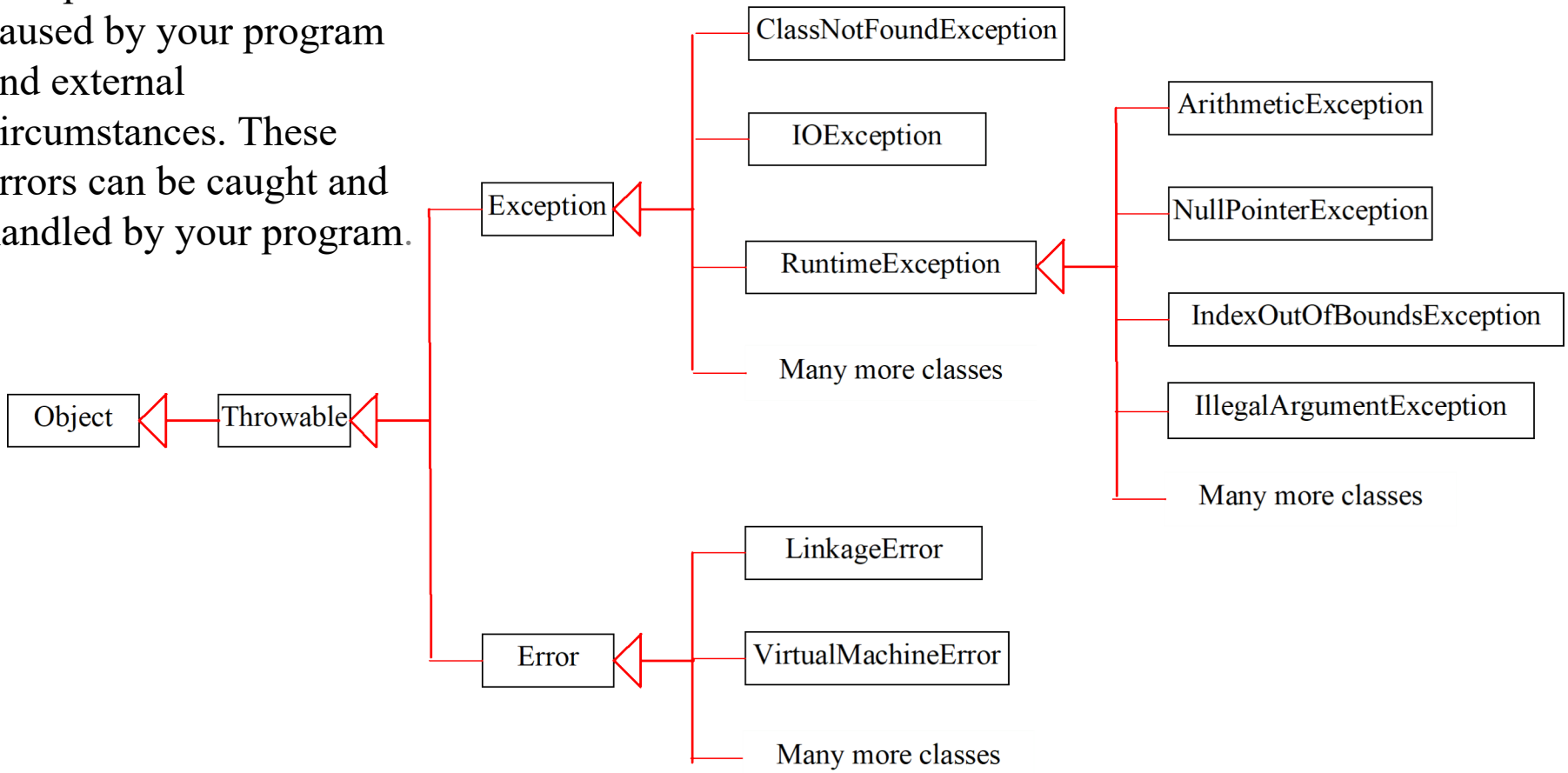
System Errors



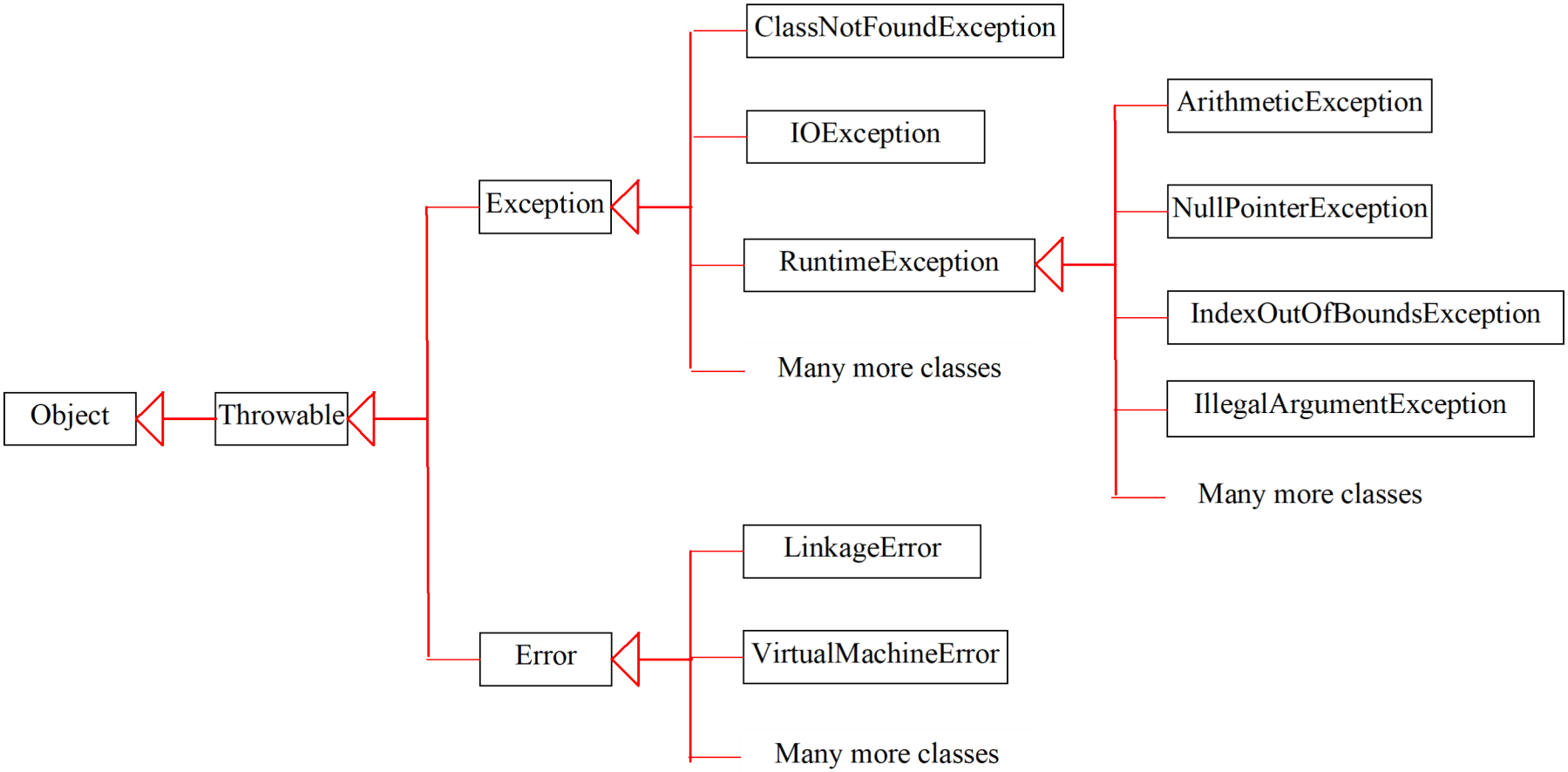
System errors are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

Exceptions

Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



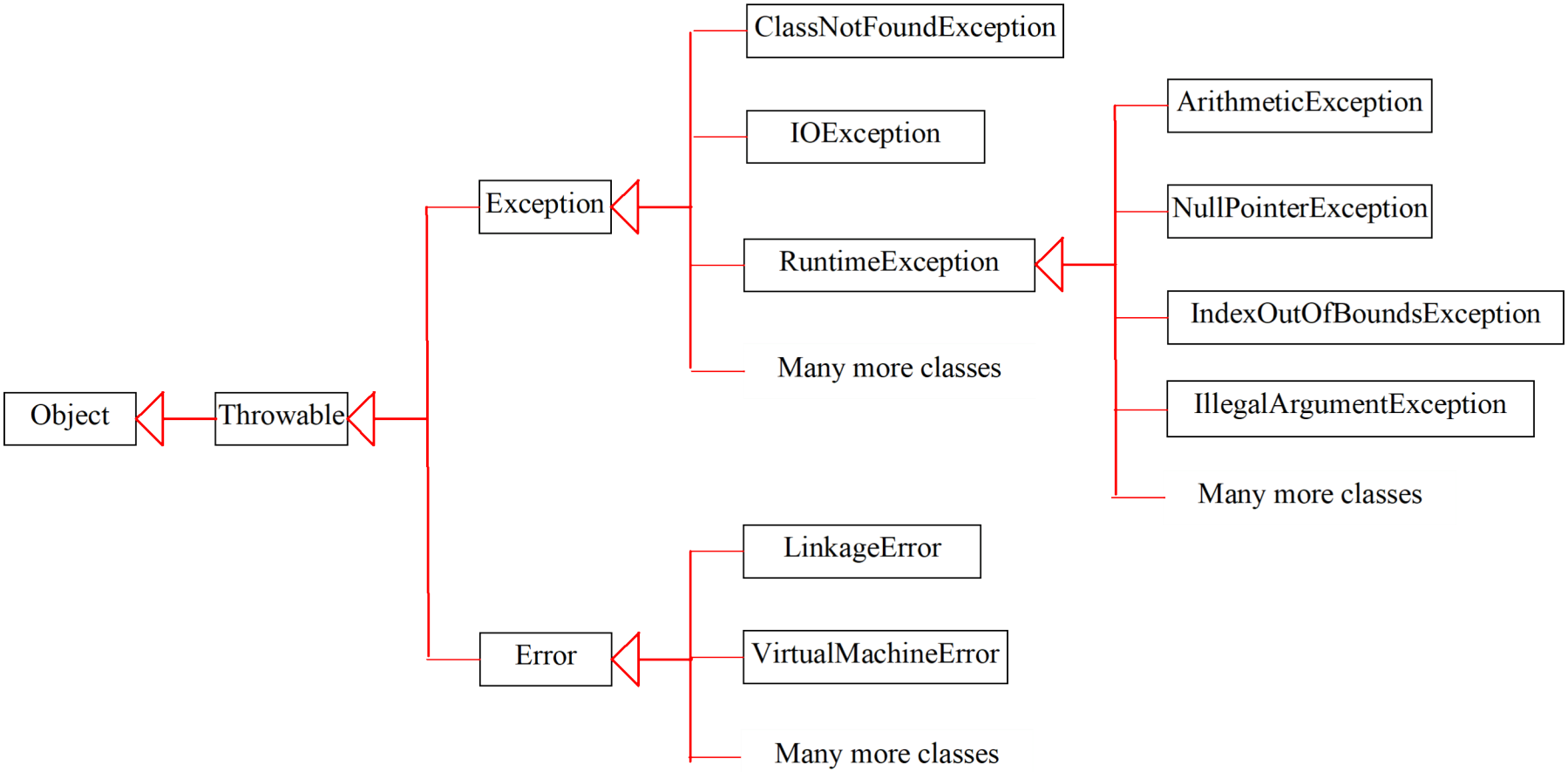
Runtime Exceptions



Checked Exceptions vs. Unchecked Exceptions

RuntimeException, Error and their subclasses are known as *unchecked exceptions*. All other exceptions are known as *checked exceptions*, meaning that the compiler forces the programmer to check and deal with the exceptions.

Unchecked Exceptions



String Handling

- String is probably the most commonly used class in Java's class library. The obvious reason for this is that strings are a very important part of programming.
- The first thing to understand about strings is that every string you create is actually an object of type String. Even string constants are actually String objects.
- For example, in the statement

  System.out.println("This is a String, too");

  the string "This is a String, too" is a String constant

String Handling

- Java defines one operator for String objects: +.
- It is used to concatenate two strings. For example, this statement
- String myString = "I" + " like " + "Java.";
  results in myString containing
  "I like Java."

String Handling

- The **String** class contains several methods that you can use. Here are a few. You can
- test two strings for equality by using **equals( )**. You can obtain the length of a string by calling the **length( )** method. You can obtain the character at a specified index within a string by calling **charAt( )**. The general forms of these three methods are shown here:
- // Demonstrating some String methods.
- class StringDemo2 {
- public static void main(String args[]) {
- String strOb1 = "First String";
- String strOb2 = "Second String";
- String strOb3 = strOb1;
- System.out.println("Length of strOb1: " +
- strOb1.length());

String Handling

```
System.out.println ("Char at index 3 in strOb1: " +
strOb1.charAt(3));
if(strOb1.equals(strOb2))

System.out.println("strOb1 == strOb2");
else
System.out.println("strOb1 != strOb2");
if(strOb1.equals(strOb3))
System.out.println("strOb1 == strOb3");
else
System.out.println("strOb1 != strOb3");
} }
```

This program generates the following output:

```
Length of strOb1: 12
Char at index 3 in strOb1: s
strOb1 != strOb2
strOb1 == strOb3
```

String Handling

Restrictions:

May not span multiple lines.

```
"This is not
a legal String."
```

May not contain a " character.

```
"This is not a "legal" String either."
```

Escape sequences

escape sequence: A special sequence of characters used to represent certain special characters in a string.

`\t`	tab character
`\n`	new line character
`\"`	quotation mark character
`\\`	backslash character

Example:

```
System.out.println("\\hello\nhow\tare \"you\"?\\\\");
```

Output:

```
\hello
how  are "you"?\\
```

Questions

What is the output of the following `println` statements?

```
System.out.println("\ta\tb\tc");
System.out.println("\\\\");
System.out.println("'");
System.out.println("\"\"\"");
System.out.println("C:\nin\the
  downward spiral");
```

Write a `println` statement to produce this output:

```
/ \ // \\ /// \\\
```

Answers

Output of each `println` statement:

```
        a       b       c
\\
'
"""
C:
in      he downward spiral
```

`println` statement to produce the line of output:

```
System.out.println("/ \\ // \\\\ /// \\\\\\");
```

Questions

What `println` statements will generate this output?

```
This quote is from
Irish poet Oscar Wilde:

"Music makes one feel so romantic
- at least it always gets on one's nerves -
which is the same thing nowadays."
```

What `println` statements will generate this output?

```
A "quoted" String is
'much' better if you learn
the rules of "escape sequences."

Also, "" represents an empty String.
Don't forget: use \" instead of " !
'' is not the same as "
```

Answers

`println` statements to generate the output:

```
System.out.println("This quote is from");
System.out.println("Irish poet Oscar Wilde:”);
System.out.println();
System.out.println("\"Music makes one feel so romantic");
System.out.println("- at least it always gets on one's nerves -");
System.out.println("which is the same thing nowadays.\"");
```

`println` statements to generate the output:

```
System.out.println("A \"quoted\" String is");
System.out.println("'much' better if you learn");
System.out.println("the rules of \"escape sequences.\"");
System.out.println();
System.out.println("Also, \"\" represents an empty String.");
System.out.println("Don't forget: use \\\" instead of \" !");
System.out.println("'' is not the same as \"");
```

THANK YOU

Java Programming

These slides contain either partial or complete content from various websites, presentations and other internet based resources. The author acknowledges the contributions of all those whose work has become part of these slides either in part or full and assures that these have been used for padagogical purposes only.